

Escalonamento de tarefas com a equação Navier-Stokes

Para essa atividade, compreendi que é necessário realizar a simulação do movimento de um fluido ao longo do tempo considerando apenas os efeitos de viscosidade da equação de Navier-Stokes. Desconsiderando pressão e forças externas, o modelo se reduz a uma equação de difusão pura.

A atividade exige o uso de diferenças finitas para discretizar o espaço (2D), a validação da estabilidade com um campo constante, a criação de uma perturbação para observar sua difusão suave e, por fim, a paralelização com OpenMP explorando as cláusulas `schedule` e `collapse`.

I. Estrutura do Grid e Alocação

Para a estruturação do Navier-Stokes, foi utilizado um grid 2D, contendo o tamanho de linhas, colunas e a própria matriz de floats. Além disso, para facilitar, foi criada uma função que aloca e inicializa um grid nos conformes da atividade: todos os pontos se inicializam com uma velocidade constante, e no centro ocorre uma perturbação.

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

#define NX 64
#define NY 64
#define NT 5000
#define DX 0.1f
#define DY 0.1f
#define DT 0.001f
#define VISC 0.1f

typedef struct {
    int nx;
    int ny;
    float **data;
} Grid2D;

Grid2D alocar_grid(int nx, int ny, float valor_base) {
    Grid2D grid = {
        .nx = nx,
        .ny = ny,
        .data = (float **)malloc(nx * sizeof(float *)),
    };

    for (int i = 0; i < nx; i++) {
        grid.data[i] = (float *)malloc(ny * sizeof(float));
    }

    for (int i = 0; i < grid.nx; i++) {
        for (int j = 0; j < grid.ny; j++) {
            grid.data[i][j] = valor_base;
        }
    }
}
```

```

    }
}

int cx = grid.nx / 2, cy = grid.ny / 2, r = 4;
for (int i = cx - r; i <= cx + r; i++) {
    for (int j = cy - r; j <= cy + r; j++) {
        grid.data[i][j] = 10.0f;
    }
}

return grid;
}

```

II. Função de Diferenças Finitas

Nessa simulação, o ato de “avançar no tempo” é dado pela função de `calcular_proximo_passo`. Essa função itera por todo o grid de entrada, e para cada ponto, utiliza os pontos em volta para atualizar a velocidade no grid de saída. É aqui que é utilizada a técnica de diferenças finitas na equação de Navier-Stokes.

Além disso, como a natureza do problema acaba sendo uma simulação temporal, cada passo pode ser paralelizado, mas não encontrei uma forma eficiente e correta de paralelizar o avanço do tempo. Dessa forma, a única parte paralelizada do programa é justamente na atualização do grid a cada iteração.

```

void calcular_proximo_passo(const Grid2D u_current, const Grid2D u_next,
                           const float nu, const float dt, const float dx,
                           const float dy) {
    const int nx = u_current.nx;
    const int ny = u_current.ny;
    const float idx2 = 1.0f / (dx * dx);
    const float idy2 = 1.0f / (dy * dy);

#pragma omp parallel for collapse(2) schedule(static, 16)
    for (int i = 1; i < nx - 1; i++) {
        for (int j = 1; j < ny - 1; j++) {
            const float laplaciano =
                (u_current.data[i + 1][j] - 2.0f * u_current.data[i][j] +
                 u_current.data[i - 1][j]) *
                idx2 +
                (u_current.data[i][j + 1] - 2.0f * u_current.data[i][j] +
                 u_current.data[i][j - 1]) *
                idy2;

            u_next.data[i][j] = u_current.data[i][j] + dt * nu * laplaciano;
        }
    }
}

```

III. Execução Principal e Simulação da Perturbação

O programa principal inicializa o fluido em calmaria (velocidade constante igual a 1.0) e gera uma perturbação energética no centro do domínio. O avanço temporal é computado alternando as referências de leitura e escrita a cada iteração para não ter necessidade de alocar memória nos hot paths. Dessa forma,

existem sempre dois grids ativos, o grid do step anterior, que é passado como readonly para a função de calcular_proximo_passo, e o grid que será atualizado atualmente, que é modificado.

Isso acontece pois não podemos atualizar o estado anterior para gerar um estado novo, precisamos, para todos os pontos, todos os estados anteriores intactos.

```
int main(void) {
    Grid2D u_velA = alocar_grid(NX, NY);
    Grid2D u_velB = alocar_grid(NX, NY);

    Grid2D u_atual = u_velA;
    Grid2D u_proximo = u_velB;

    printf("Velocidade inicial no centro: %.2f\n", u_atual.data[NX / 2][NY / 2]);

    double tempo_inicio = omp_get_wtime();

    for (int t = 0; t < NT; t++) {
        calcular_proximo_passo(u_atual, u_proximo, VISC, DT, DX, DY);

        Grid2D temp = u_atual;
        u_atual = u_proximo;
        u_proximo = temp;
    }

    double tempo_fim = omp_get_wtime();

    printf("Velocidade final no centro apos %d passos: %f\n", NT, u_atual.data[NX / 2][NY / 2]);
    printf("Tempo total gasto: %f segundos\n", tempo_fim - tempo_inicio);

    liberar_grid(u_velA);
    liberar_grid(u_velB);

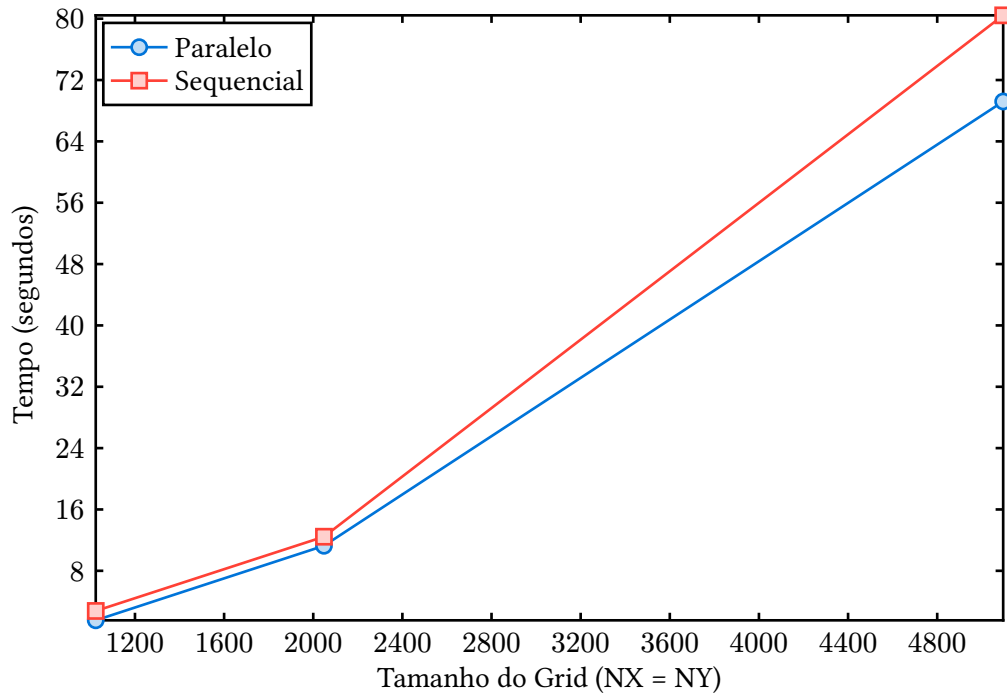
    return 0;
}
```

IV. Comparação paralela e sequencial

Devido a natureza do problema não ser inteiramente paralelizável, aliado ao fato de que meu notebook utilizado nos testes tem apenas 2 processadores e 4 threads, fez com que os resultados não fossem tão satisfatórios quanto esperado. Devido a isso, será interessante ver como esse programa se comportará com mais cores pelo NPAD.

Para os testes, note que a versão sequencial é igual a versão paralela, removendo apenas a diretiva do openmp.

IV. I. Comparativo de Desempenho: Paralelo vs Sequencial



V. Collapse e Schedule

Foi testado a utilização de diretivas de `collapse(2)`. Mas a performance degradou significativamente, o fenômeno pode ser explicado pela invalidação de cache entre as threads, visto que Temos uma estrutura de matriz row-first e o cache que obedece a localidade espacial, então em linhas gerais, é mais interessante manter uma linha inteira por thread, ou pelo menos uma seção inteira da linha para uma thread.